Description of Design

Explanation of neuron.v file

The neuron.v file contains the modules neuron_train and snn_train_1.

The neuron_train module has an internal parameter v_membrane and it takes an excitation as another input (along with clock and reset). The excitation input is determined by the snn_train_1 module. The outputs of the module are the output_spike_sr 3-bit shift register, the output_spike register, the v_membrane 7-bit register, and the 7-bit spike_count register. The neuron_train module initializes the membrane voltage to V_rest, the output_spike register to 0, the output_spike_sr shift register to 3 bits of 0, and the spike_count register to 0. If there is an output spike or an inhibition signal (which happens when the other neuron fires but the current neuron doesn't fire), the membrane voltage is set back to rest, the output spike is set to 0, and the spike shift register shifts over and adds a 0. If the next membrane voltage will be over the threshold, I update the membrane voltage, output the spike, update the spike count, and update the shift register. This makes it work so that the next clock cycle, the fact that the output spike is a 1 will cause the membrane voltage to go back to 0, which leaves the output spike high for one clock cycle (as per the homework instructions). If the next clock cycle will take the membrane voltage below the resting voltage, the membrane voltage in the next clock cycle will just be the rest voltage, and output spike will be a 0. If none of these special cases are met, the membrane voltage will be updated based on the learning rule, and there is no output spike. I ensured that all of these were non-blocking operations.

The snn_train_1 module takes the 25 input spikes, clock, reset, train, and train enable. It outputs the output spikes, membrane voltages, output shift registers, and spike counts of neurons 1 and 2 (from the output layer). I declare the values and then calculate the excitation using the rule outlined in the homework assignment. Then, I transfer that result to the neuron module. This module is parametrized by the paths to the two initial weights files.

I chose a bit resolution of 7 bits (unsigned) for all of my registers/wires that weren't binary values. This handles the maximum membrane voltage that I would see, which prevents integer overflow.

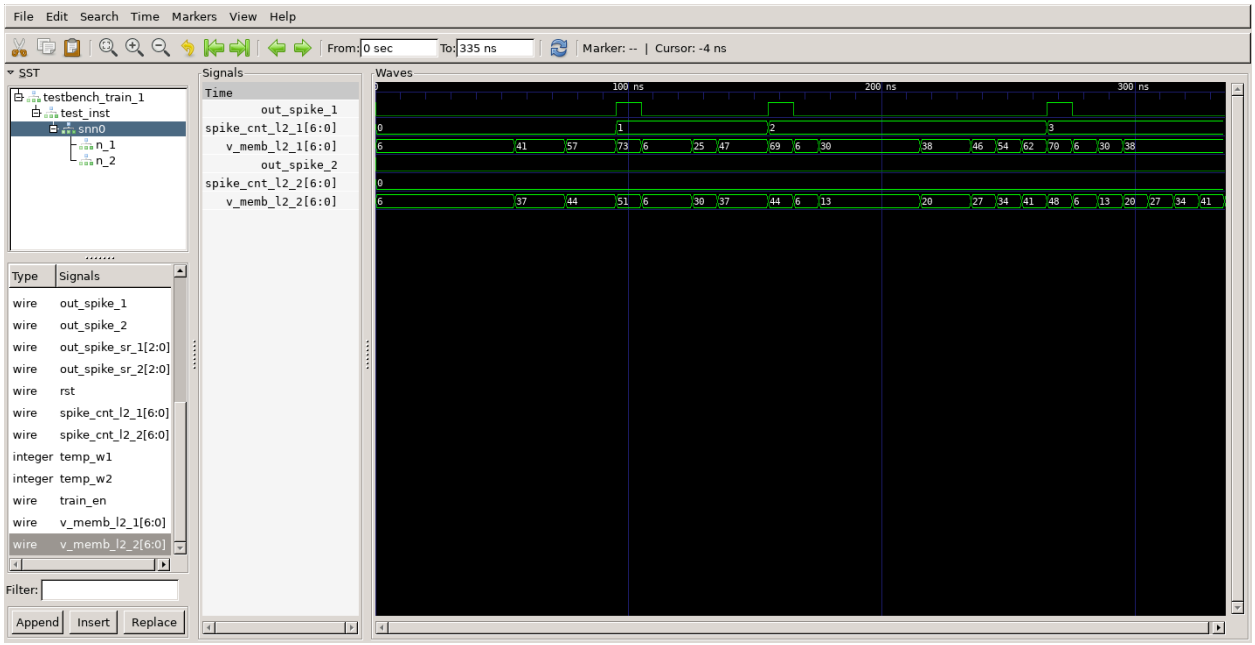
Explanation of tbs.v file

I made a base testbench for training and created instances of it for each digit in the training and testing datasets.

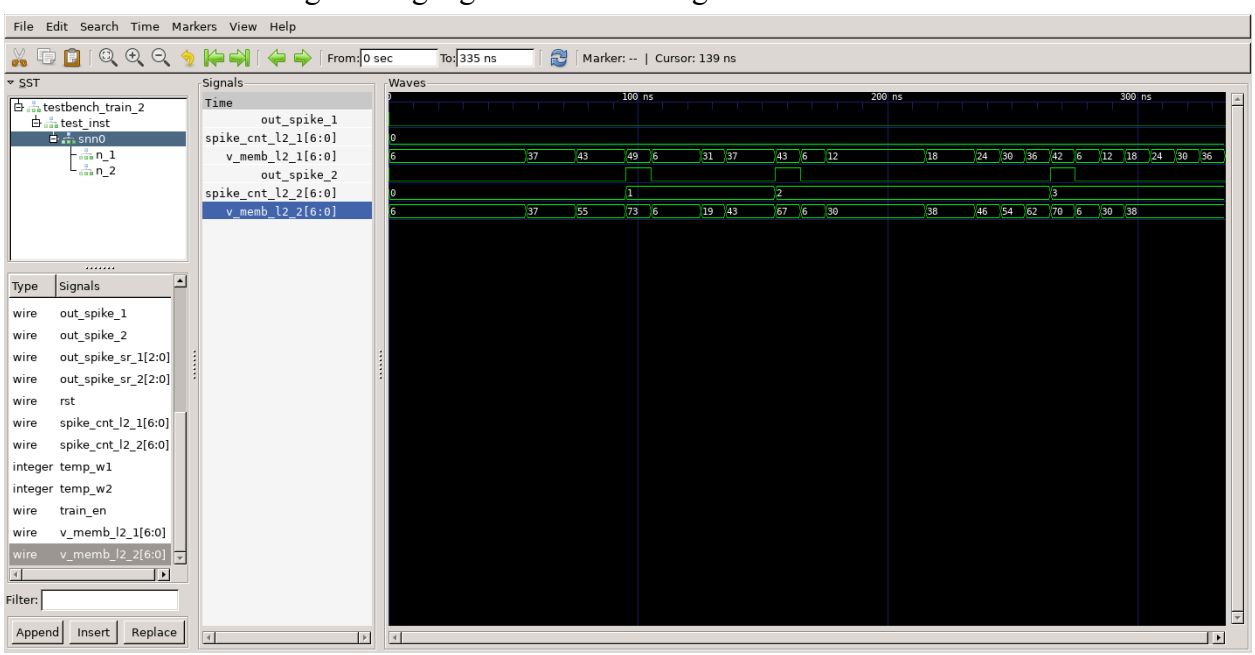
Each testbench is parametrized by paths for the initial and final weights paths for each neuron, the file path for the spikes, the output waveform display path, and a variable to indicate training or testing. Then, I have a wire for each variable in the snn_train_1 module, and I hook up a snn_train_1 module for each neuron. Next, I initialize the clock to reset its polarity every 5 timesteps, corresponding to a 10ns period. After that, I have an initial begin block. I set up waveform dumping based on the parameters for the specific testbench. I read in the spike file. I set up the input bus and the clock, and I assert reset for 2 clock cycles before releasing it. Then, I iterate through each bit in the spike train and set the input bus based on that clock cycle of inputs

from the spike matrix. After that, I clear the input bus, wait 70 timesteps, and write the trained paths to output files, if training is enabled. I made 4 testbenches from this template.
Explanation of running the Verilog files

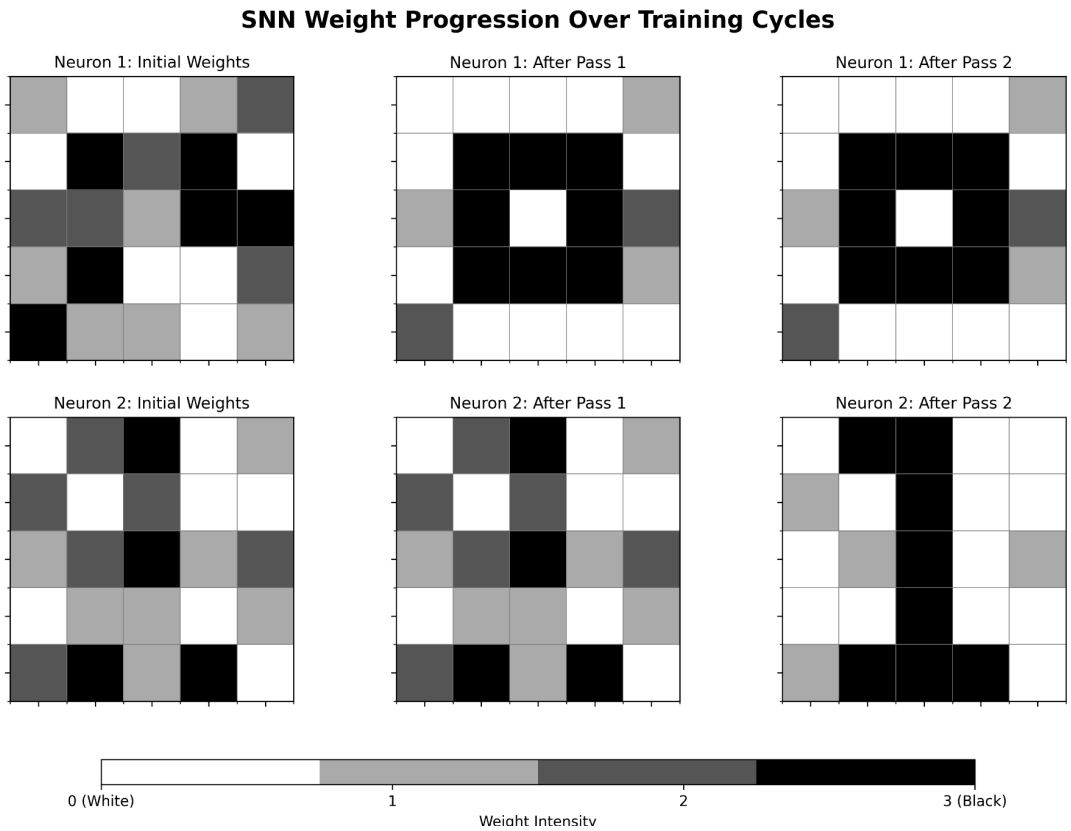
Results



Waveforms after running training algorithm on training 0

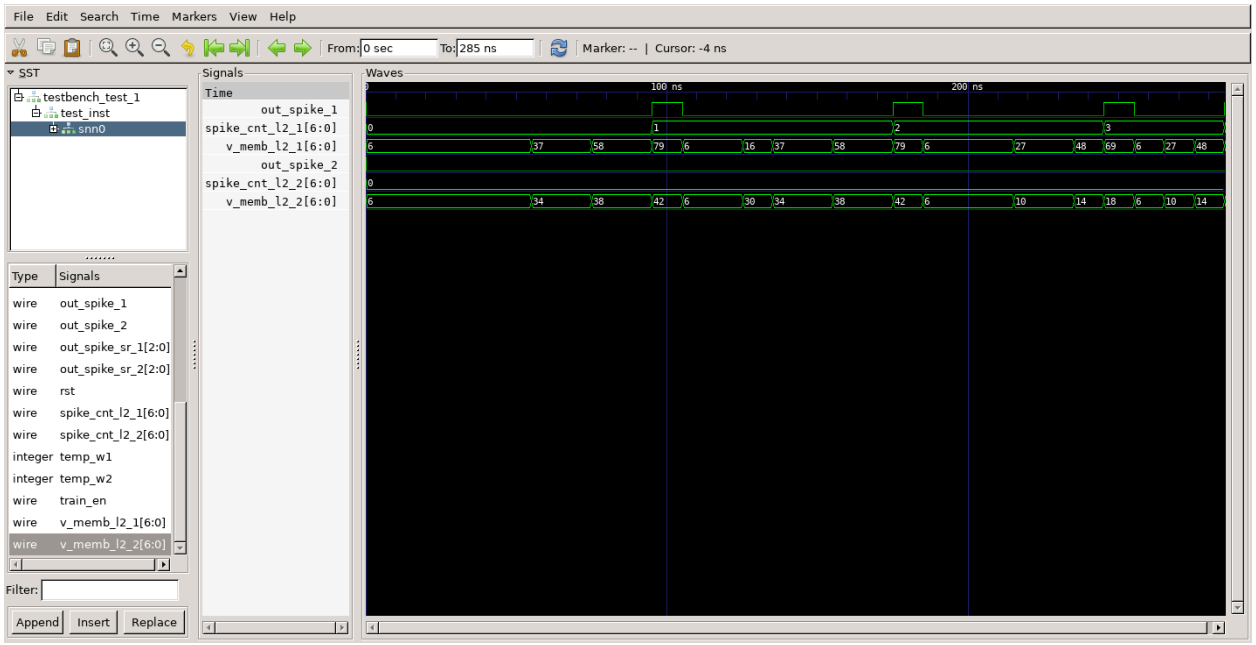


Waveforms after running training algorithm on training 1

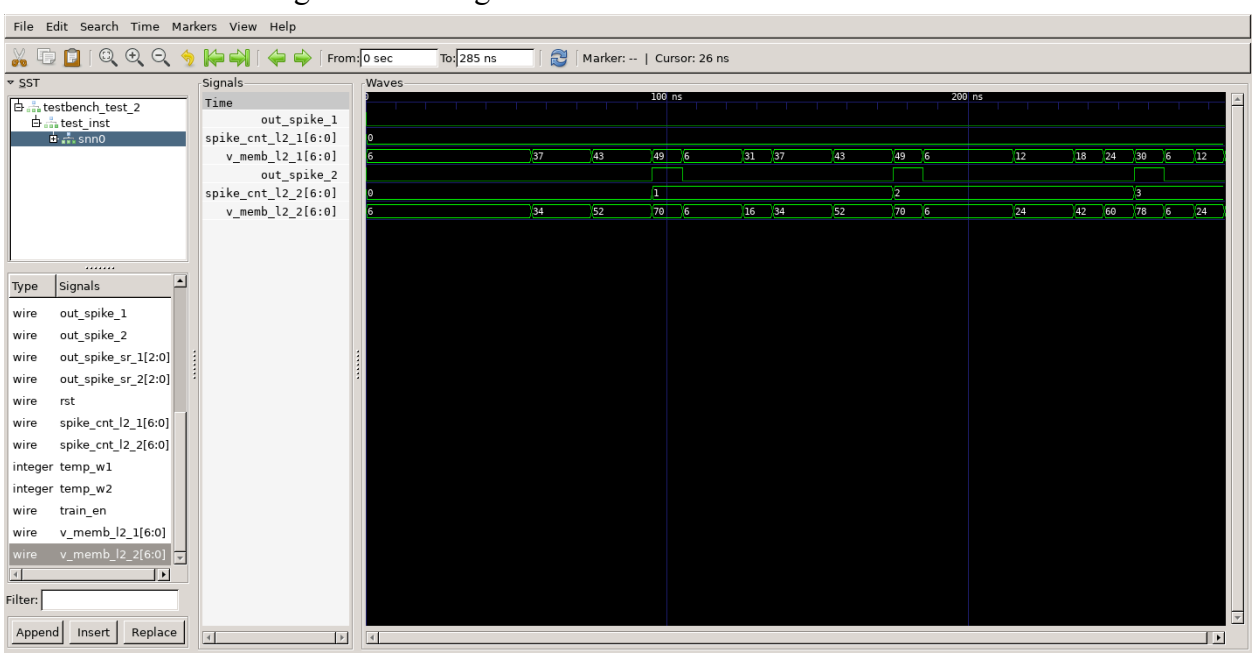


Progression of weights through the two training cycles

My weight maps show that the first neuron has high weights for the positions that align with the shape of the 0 from the training data and that the second neuron has high weights for the positions that align with the shape of the 1 from the training data. From this, I can assume that neuron 1 detects the digit 0 and that neuron 2 detects the digit 1. Since I only ran the training algorithm once for each digit, and there was only one example for each member of the training and testing datasets, the weights for the pixels not corresponding to the locations of the black pixels could only be depressed by so much. That's why there are some pixels that are still light gray around the 1, and some pixels that are dark gray and light gray around the 0 (in the weight maps for neurons 1 and 2 after both passes of training). It's also important to note that neuron 2 did not fire at all during pass 1, and neuron 1 did not fire at all during pass 1; this makes sense, as neuron 2 did not have any weight updates during pass 1, and neuron 1 did not have any weight updates during pass 1. In pass 1, neuron 1 fires three times and neuron 2 fires 0 times. In pass 2, neuron 2 fires three times and neuron 1 fires 0 times.



Waveforms from testing on the testing 0 value



Waveforms from testing on the testing 1 value

In pass 1 of testing, neuron 1 fires 3 times and neuron 2 fires 0 times. This suggests that neuron 1 has learned to recognize the digit 0, and other patterns that look similar to digit 0, while not producing an output when a digit like digit 1 is presented. In pass 2 of testing, neuron 1 fires 0 times and neuron 2 fires 3 times. This suggests that neuron 2 has learned to recognize the digit 1, and other patterns that look similar to digit 1, while not producing an output when a digit like digit 0 is presented.

This learning process is enabled through spike timing-dependency plasticity (STDP). In STDP, if there is a presynaptic spike followed by a postsynaptic spike (within 2 timesteps, as defined by this homework), the weight between the neuron that generated the presynaptic spike and the neuron that generated the postsynaptic spike is increased. Our weight values are constrained between 0 and 3 in this assignment. If the time gap between the presynaptic spike and the postsynaptic spike was 1, the weight is increased by 2. If the time gap between the presynaptic spike and the postsynaptic spike was 2, the weight is increased by 1. This is long-term potentiation since the weight increased. Conversely, if the time gap between the presynaptic spike and the postsynaptic spike was -1, the weight is decreased by 2. If the time gap between the presynaptic spike and the postsynaptic spike was -2, the weight is decreased by 1.

In this assignment, we were provided a 20-bit sequence to represent the color black and a different sequence to represent the color white. These sequences determine the firing of each neuron. The membrane potentials are accumulated with an integrate-and-fire rule (no leak), and the concentration of the bit 1 is higher in the black pixel than in the white pixel, which suggests that the SNN can learn which pixels are dark since there is more firing from that spike pattern. The threshold for the neuron to output a spike is 65, meaning that the pixels with the higher trained weights can take the same input spike and produce a greater increase in the membrane voltage. If the testing image has a high frequency of spikes on the pixels with the higher weights (which is the case because of how the black pixel has more 1 bits than in the white pixel), it has a higher chance of increasing the membrane potential to and above the threshold voltage, resulting in the neuron firing for the testing image from the class that it was supposed to identify.

This brings up some questions that we will attempt to address in our final project. We will examine the effect of learning from pre-post-pre and post-pre-post spike sequences since the LTP from the pre-post part of the pre-post-pre would be canceled by the following post-pre of the pre-post-pre. We will investigate the effect of changing the frequency of 0s and 1s in the bit patterns for each pixel. We will also look at all-to-all vs. nearest neighbor learning methods. In this assignment, I assumed that we would implement nearest neighbor learning since this would prevent multiple LTP and/or multiple LTD events from happening in the same clock cycle.

```
if (out_spike_sr_1[0]) begin
    // LTP: check if pre fired 1 or 2 cycles ago
    if      (pre_spike_sr[j][1]) dw1 = dw1 + 2;
    else if (pre_spike_sr[j][2]) dw1 = dw1 + 1;
end
// Only evaluate LTD when a pre-spike occurs and post fired recently
if (pre_spike_sr[j][0]) begin
    if      (out_spike_sr_1[1]) dw1 = dw1 - 2;
    else if (out_spike_sr_1[2]) dw1 = dw1 - 1;
end
```

This code block shows that implementation. Specifically, the else if statements prevent all-to-all weight updates. Note the lack of an else before the second highest-level if. This ensures that there can be LTP and LTD in the same clock cycle.